

Node.js + Express + Sequelize + PostgreSQL — API Assignment

Overview

Build a **RESTful API** using **Node.js**, **Express**, **Sequelize**, and **PostgreSQL** that manages Products and Users.

The application should include:

- JWT Authentication
- Role-Based Access Control (RBAC)
- Product Management (CRUD, search, sort, pagination)
- File uploads (images, CSV)
- Scheduled notifications
- Error & warning logging

Database Models

1. User Model

Field	Type	Description
id	UUID / Auto Increment	Primary key
name	STRING	Full name
email	STRING (unique)	User email
password	STRING (hashed)	Encrypted password
role	ENUM('admin', 'user')	Defines access level
createdAt	DATETIME	Creation time
updatedAt	DATETIME	Update time

2. Product Model

Field	Type	Description
id	UUID / Auto Increment	Primary key

name	STRING	Product name
description	TEXT	Product details
price	DECIMAL(10,2)	Product price
rating	FLOAT	Product rating
image	STRING (path/url)	Product image
published	BOOLEAN	Published status
userId	UUID (FK → User.id)	Owner user
createdBy	UUID (FK → User.id)	Admin who created
createdAt	DATETIME	Creation time
updatedAt	DATETIME	Update time

Authentication & Authorization

- **JWT-based Authentication** for all APIs (token in headers)
- **Roles:**
 - **Admin**
 - Can create, update, delete users & products
 - Can view all products with filters, pagination
 - Can perform bulk upload
 - **User**
 - Can view own products
 - Can create, update, delete their own products only

API Endpoints

User APIs

Method	URL	Action	Auth	Role
POST	/api/users	Add new user	JWT	Admin only
PUT	/api/users/:id	Update user by ID	JWT	Admin
DELETE	/api/users/:id	Delete user	JWT	Admin

Product APIs

Meth od	URL	Action	Au th	Role
GET	/api/products	Get all products (with pagination, search, sort)	JW T	Admin/U ser
GET	/api/products/:id	Get product by ID	JW T	Admin/U ser
GET	/api/products/user/:userId	Get all products by user ID	JW T	Admin/U ser
POST	/api/products	Add new product	JW T	Admin/U ser
POST	/api/products/:userId	Add new product by user ID	JW T	Admin
PUT	/api/products/:id	Update product by ID	JW T	Admin/U ser
PUT	/api/products/:userId/:productId	Update product by UserID	JW T	Admin
DELE TE	/api/products/:id	Delete product by ID	JW T	Admin/U ser
DELE TE	/api/products	Delete all products	JW T	Admin
GET	/api/products/published	Get all published products	JW T	All
GET	/api/products?name=<query>	Search products by name	JW T	All

Additional Features

Filtering & Sorting

- Query params:

GET /api/products?name=phone&sort=price&order=asc&page=1&limit=10

Pagination

- Use Sequelize limit and offset
- Return total count, current page, and total pages

Bulk Upload (CSV)

- Endpoint: POST /api/products/upload
- Upload a CSV file with columns: name, description, price, rating, published
- Parse using csv-parser and bulk insert using Sequelize
- Minimum 1000 products using bullmq (queue)

File Upload (Images)

- Multiple images allowed
- Max size per image: 10MB
- Use multer for uploads
- Store file paths in database
- Open-source bucket

Download All Products

- Endpoint: GET /api/products/download
- Return all product data as CSV or JSON

Scheduler (Notifications)

- Use node-cron
- Schedule daily/weekly task to send notifications to users about products added by admin on their behalf
- Store notification logs in a separate table or log file

Monthly Consolidated Report (Admin)

- Endpoint: GET /api/admin/summary
- Return:
 - Number of users added (per month)
 - Number of products added (per month)
 - Use GROUP BY month on createdAt

Logging

- Use winston or morgan
- Log levels: error, warn, info
- Separate files for error logs and request logs

- Activity logs user actions

Tech Stack

- **Backend:** Node.js, Express.js
- **ORM:** Sequelize
- **Database:** PostgreSQL
- **Auth:** JWT
- **File Upload:** Multer
- **Scheduler:** node-cron
- **Logging:** Winston / Morgan
- **CSV Parsing:** csv-parser

Folder Structure (suggested)

project/

```
|
|├── src/
| |
| |├── config/
| | |
| | |├── db.config.js
| | |└── auth.config.js
| |
| |├── middleware/
| | |
| | |├── authJwt.js
| | |├── roleCheck.js
| | |└── upload.js
| |
| |├── models/
| | |
| | |├── index.js
| | |├── user.model.js
| | |└── product.model.js
| |
| |├── controllers/
| | |
| | |├── user.controller.js
| | |└── product.controller.js
| |
| |├── routes/
| | |
| | |├── user.routes.js
| | |└── product.routes.js
| |
| |├── services/
| | |
| | |├── csv.service.js
| | |└── notification.service.js
| |
| └── utils/
```

```
|   ├── logger.js
|   └── responseHandler.js
|── .env
|── server.js
└── package.json
```

Bonus (Optional)

- Add Swagger documentation for all APIs
- Add unit tests using Jest
- Dockerize the application.